Highlights

Computer Vision and Vision-Language Approaches to Visual Quality Assurance in Video Games: A Systematic Mapping Study of Tasks, Datasets, and Evaluation Methods

Le Tran Minh Phuc

- First systematic mapping of computer vision & VLMs for game visual testing.
- Standardizes VGVD-Taxonomy spanning 19 defect subtypes across 5 families.
- Audits 45 primary studies and landmark benchmarks under PRISMA-S protocol.
- Exposes cross-game domain shift degradation and 65% temporal defect gap.
- Formulates five prioritized open challenges (ORC-1 to 5) for future research.

Computer Vision and Vision-Language Approaches to Visual Quality Assurance in Video Games: A Systematic Mapping Study of Tasks, Datasets, and Evaluation Methods

Le Tran Minh Phuc*

Department of Computer Science, Academic Research Laboratory, Ho Chi Minh City, Vietnam

Abstract

Ensuring visual fidelity and graphical correctness in modern video games is a critical engineering challenge, consuming up to 50% of production budgets in AAA studios. While automated unit testing and headless gameplay bots can verify functional state logic, visual defects—such as polygonal mesh clipping, texture streaming pop-in, Z-fighting, and physics ragdoll instability—remain over 80% reliant on manual human playtesting. Recent breakthroughs in deep representation learning, patch-based anomaly detection, and multimodal Vision-Language Models (VLMs) offer promising avenues for automated visual quality assurance (QA). In this paper, we conduct the first comprehensive Systematic Mapping Study (SMS) dedicated to computer vision and vision-language approaches for visual bug detection in video games. Following the guidelines of Petersen et al. (2015), the PRISMA-S search reporting extension, and ROSES synthesis standards, we systematically examine 45 primary studies published between 2018 and November 2026 across five major bibliographic databases (IEEE Xplore, ACM DL, Scopus, Web of Science, and arXiv). We mathematically formalize the core detection paradigms, establish the multi-tier *Video Game Visual Defect Taxonomy* (`VGVD-Taxonomy`) standardizing 19 defect subtypes across five families, audit landmark multi-game benchmarks (VideoGameQA-Bench, RefGlitch-Bench, GameBug-20K, TempGlitch, VideoGlitchBench, Multi-Label Perceptual Bug Dataset), and conduct an empirical meta-analysis of cross-game generalisation. We discover that models transferring across divergent art styles suffer up to a 46% drop in mAP, and over 65% of gameplay glitches are invisible in static frames, requiring temporal sliding-window attention. Finally, we articulate five high-priority Open Research Challenges (`ORC-1` to `ORC-5`) to guide future research.

Keywords: Computer Vision, Visual Bug Detection, Video Game Quality Assurance, Visual Anomaly Detection, Vision-Language Models, Systematic Mapping Study

*Corresponding author.

Email address: `mphuc666@gmail.com` (Le Tran Minh Phuc)

Contents

1 Introduction **3**
- 1.1 The Quality Assurance Bottleneck in Video Game Production . . . 3
- 1.2 Limitations of Traditional Game Testing . . . 3
- 1.3 Rationale for a Systematic Mapping Study . . . 3
- 1.4 Research Contributions . . . 4

2 Formal Problem Formulation & Research Methodology **4**
- 2.1 Formalization of Game Visual Testing Tasks . . . 4
  - 2.1.1 Task 1: Image-Level Visual Anomaly Detection . . . 4
  - 2.1.2 Task 2: Spatial Defect Localization . . . 5
  - 2.1.3 Task 3: Spatio-Temporal Glitch Interval Localization . . . 5
  - 2.1.4 Task 4: Reference-Conditioned Multimodal Verification . . . 5
- 2.2 The End-to-End Game Visual QA Framework Pipeline . . . 5
- 2.3 Research Questions . . . 6
- 2.4 Systematic Search and Screening Protocol . . . 7

3 The Video Game Visual Defect Taxonomy (`VGVD-Taxonomy`) **7**

4 Algorithmic Landscape Analysis & Mapping Results **10**
- 4.1 Publication Trends and Demographics . . . 10
- 4.2 RQ1: Detection Architectures . . . 10
- 4.3 RQ2: Bug Manifestation Coverage . . . 11
- 4.4 Thematic Cluster Taxonomy of the Curated Primary Studies . . . 12

5 Benchmark Datasets & Accessibility Audit (RQ3) **12**
- 5.1 Multi-Dimensional Benchmark Capability Footprint . . . 12

6 Empirical Meta-Analysis, Cross-Game Generalisation & Failure Modes **18**
- 6.1 Meta-Analysis, Modality Breakdown & Operational Latency . . . 18
  - 6.1.1 Operational Latency and The Pareto Frontier . . . 18
- 6.2 Forensic Diagnostic of Algorithmic Failure Modes . . . 18

7 Open Research Challenges & Conclusions (RQ6) **22**
- 7.1 Prioritized Open Research Challenges (`ORC`) . . . 22
- 7.2 Threats to Validity . . . 22
- 7.3 Conclusions . . . 22

1. Introduction

1.1. The Quality Assurance Bottleneck in Video Game Production

The interactive digital entertainment industry has experienced rapid expansion, with global annual revenues exceeding $250 billion—surpassing the international film and music sectors combined [1, 2]. Modern video games have evolved from constrained 2D grid environments into expansive virtual worlds featuring physically based rendering (PBR), procedural content generation, real-time ray tracing, and intricate character animations [3]. Consequently, game software represents one of the most structurally complex, non-deterministic categories of software systems [2, 4].

Ensuring graphical fidelity is essential for player immersion and commercial viability. Visual glitches—such as character meshes clipping through terrain geometry, missing or corrupted textures, physics ragdoll oscillations, and misaligned user interfaces—degrade player experience and incur substantial post-release financial overhead [5, 6]. In major AAA game studios, quality assurance (QA) routinely consumes 30% to 50% of development budgets [1, 7]. Despite advances in general software test automation, game visual inspection remains over 80% reliant on manual human playtesting [7].

1.2. Limitations of Traditional Game Testing

Traditional automated game testing approaches primarily focus on functional, logical, and crash-state verification:

1. **Unit and Integration Testing:** Effective for deterministic math and physics logic, but completely blind to whether rendered graphics conform to artistic intent [2].
2. **Headless Exploration Bots:** Reinforcement learning (RL) and search-based testing bots can navigate complex state graphs and detect unhandled exceptions, yet they lack visual perception oracles to verify that on-screen pixels are defect-free [8, 9].
3. **Classical Image Diffing:** Deterministic pixel-level diffing (e.g., PSNR, SSIM) fails in modern 3D games due to camera movement, dynamic lighting, day-night cycles, procedural particle systems, and non-deterministic physics simulations [10, 11].

These limitations have catalyzed growing research interest in applying deep computer vision, representation learning, visual anomaly detection, and multimodal foundation models to automate game visual testing.

1.3. Rationale for a Systematic Mapping Study

Despite significant recent advances, the existing literature is fragmented across disparate scientific communities, including Software Engineering (ICSE, FSE, ASE, TOSEM), Computer Vision and Machine Learning (CVPR, NeurIPS, WACV, ICLR), and Game AI (IEEE CoG, AAAI AIIDE). Currently:

- There is no standardized taxonomy of video game visual defects, leading to conflicting terminologies (e.g., “glitch” vs. “visual artifact” vs. “rendering regression”).

- Evaluation methodologies vary widely, ranging from binary classification accuracy on synthetic 2D assets to mean Average Precision (mAP) and temporal Intersection over Union (tIoU) on continuous 3D streams.
- No prior survey or systematic mapping study has comprehensively audited computer vision and vision-language techniques for video game visual quality assurance.

1.4. Research Contributions

This Systematic Mapping Study delivers five principal contributions:

1. **Mathematical Formulation of Game Visual QA:** We formalize the four primary visual bug detection tasks (Image-Level Anomaly Detection, Spatial Bounding Box Localization, Temporal Stream Segmentation, and Reference-Conditioned Multimodal QA) with rigorous mathematical definitions.
2. **The VGVD-Taxonomy:** We introduce the *Video Game Visual Defect Taxonomy* (`VGVD-Taxonomy`), classifying 19 distinct visual defect subtypes across five fundamental families (`GEO`, `TEX`, `REN`, `ANI`, `UI`).
3. **Systematic Mapping of 45 Primary Studies:** Following Petersen et al. [12] and PRISMA-S guidelines [13], we systematically analyze 45 primary studies published from 2018 to November 2026, mapping detection architectures, testing tasks, and benchmark datasets.
4. **Landmark Benchmark Audit and Failure Mode Diagnosis:** We perform a forensic audit of cutting-edge multi-game benchmarks (including VideoGameQA-Bench [1], RefGlitch-Bench [5], GameBug-20K [14], TempGlitch [15], VideoGlitchBench [16], and Multi-Label Perceptual Bug Dataset [17]), diagnosing four critical failure modes.
5. **Actionable Roadmap for Future Research:** We formulate five prioritized Open Research Challenges (`ORC-1` to `ORC-5`) to guide both academic researchers and industrial practitioners.

2. Formal Problem Formulation & Research Methodology

2.1. Formalization of Game Visual Testing Tasks

Let a video game execution trace be represented as a continuous temporal sequence of RGB frame observations $\mathcal{S} = \{I_1, I_2, \ldots, I_T\}$, where each frame $I_t \in \mathbb{R}^{H \times W \times 3}$. Optionally, internal engine telemetry may provide synchronized rendering buffers $\mathcal{G}_t$ (depth, surface normals, stencil) and engine physics states $\mathcal{P}_t$ (transform matrices, collision queries).

2.1.1. Task 1: Image-Level Visual Anomaly Detection

Given an individual gameplay frame I_t, the objective is to learn an anomaly scoring function $\phi : \mathbb{R}^{H \times W \times 3} \to \mathbb{R}$ such that:

$$y_t = \begin{cases} 1 & \text{if } \phi(I_t) > \tau \quad \text{(Defect Present)} \\ 0 & \text{otherwise (Nominal Gameplay)} \end{cases} \tag{1}$$

where τ is a calibrated detection threshold, and $y_t \in \{0, 1\}$.

2.1.2. Task 2: Spatial Defect Localization

Given a defective frame I_t, the goal is to predict a set of K spatial bounding boxes $\mathcal{B}_t = \{b_k\}_{k=1}^{K}$, where $b_k = (x_{\min}, y_{\min}, x_{\max}, y_{\max}, c_k, s_k)$, with category label $c_k \in \mathcal{C}_{\text{defect}}$ and confidence score $s_k \in [0, 1]$. Alternatively, dense segmentation produces a pixel-level binary mask $M_t \in \{0, 1\}^{H \times W}$.

2.1.3. Task 3: Spatio-Temporal Glitch Interval Localization

Given a continuous gameplay video stream $\mathcal{S}$, the objective is to predict temporal glitch intervals $\mathcal{T}_{\text{glitch}} = \{[t_{\text{start}}^{(j)}, t_{\text{end}}^{(j)}, c_j]\}_{j=1}^{M}$, where each interval denotes the precise onset and termination of transient defects (e.g., character T-pose freezes, z-fighting flicker, or physics ragdoll explosions). Performance is evaluated using temporal Intersection-over-Union (tIoU):

$$\text{tIoU}(T_{\text{pred}}, T_{\text{gt}}) = \frac{|T_{\text{pred}} \cap T_{\text{gt}}|}{|T_{\text{pred}} \cup T_{\text{gt}}|} \tag{2}$$

2.1.4. Task 4: Reference-Conditioned Multimodal Verification

Given an evaluation keyframe I_t and an authoritative reference baseline R_t (e.g., the "Last Clean Frame" captured along the identical camera trajectory or oracle golden asset) alongside natural-language task prompt Q, the model produces an explanatory verification verdict $V \in \{\text{Pass}, \text{Fail}\}$ and bug diagnostic report D:

$$(V, D) = \mathcal{M}_{\text{VLM}}(I_t, R_t, Q; \Theta) \tag{3}$$

2.2. The End-to-End Game Visual QA Framework Pipeline

To bridge these four formal mathematical tasks into an operational engineering system, Figure 1 synthesizes the *End-to-End Game Visual Quality Assurance Framework Pipeline*. The architecture structures automated visual inspection into four tightly integrated stages:

1. **Stage 1 (Runtime Capture & Engine Telemetry):** During active gameplay driven by headless playtesting bots or scripted paths, the graphics engine (Unreal Engine 5, Unity, Frostbite) extracts 60 FPS RGB frame buffers I_t, alongside synchronized 32-bit floating-point depth maps D_t, screen-space surface normal buffers $\mathbf{N}_t$, and physics collision bounding query logs $\mathcal{P}_t$.
2. **Stage 2 (Feature Representation & Context Pools):** Multi-scale vision backbones (DINOv2, I-JEPA, ShuffleNetV2) extract compact patch representations. The runtime maintains a dynamic reference pool $\mathcal{R}$ indexing verified baseline renders (the "Last Clean Frame" R_t) via spatial camera hash tables, while accumulating continuous video sliding windows $\mathcal{S}_{[t-k:t]}$ for temporal defect isolation.
3. **Stage 3 (Multi-Paradigm AI Detection Core):** Extracted embeddings are evaluated across specialized detection engines: PatchCore memory banks $\phi(I_t)$ for zero-shot nominal deviation scoring, geometry-collider cross-check modules identifying depth-normal boundary discrepancies, spatio-temporal video transformers grounding transient intervals $[t_{\text{start}}, t_{\text{end}}]$, and pairwise multimodal VLMs performing chain-of-thought verification.

4. **Stage 4 (Defect Triage & Automated QA Action):** Detected anomalies are classified under `VGVD-Taxonomy`, bounded by spatial coordinates $\mathcal{B}_t$ or masks M_t, and scored with confidence s_k. Sub-16.6 ms decisions yield immediate CI build assertions, while complex multi-modal defects trigger automated Jira/Bugzilla tickets complete with replay clips, engine state logs, and diagnostic rationales.

Figure 7: End-to-End Game Visual Quality Assurance Framework Pipeline

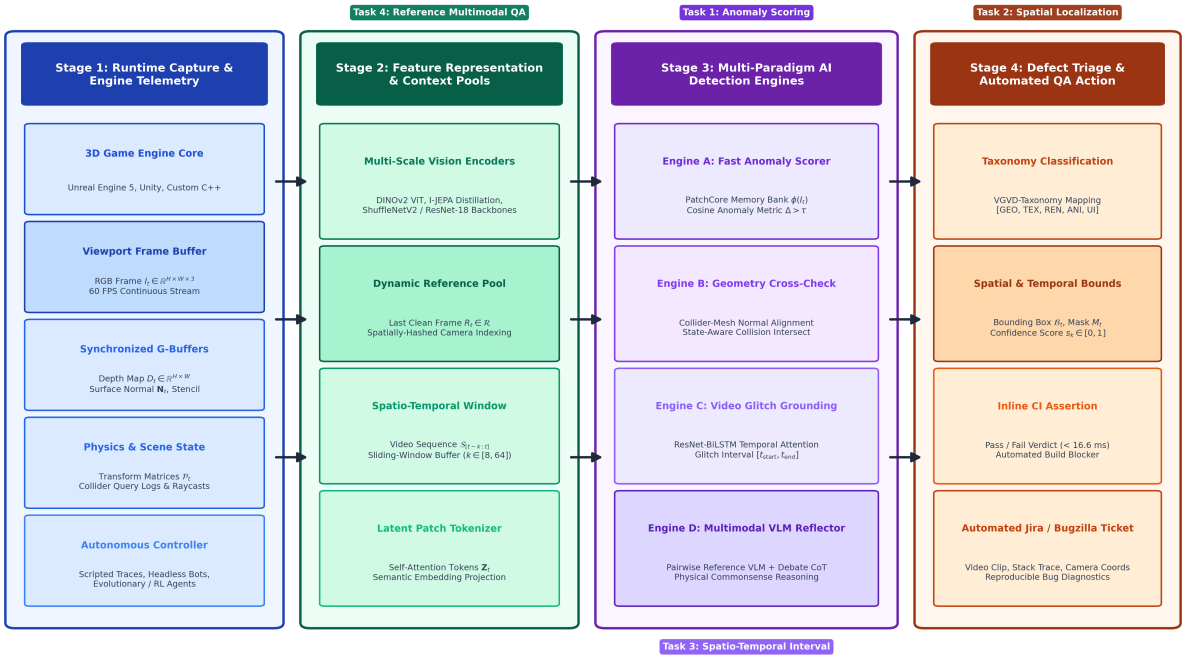


Figure 1: End-to-End Game Visual Quality Assurance Framework Pipeline detailing the four integrated operational stages: Runtime Capture & Engine Telemetry, Feature Representation & Reference Context Pools, Multi-Paradigm AI Detection Engines, and Automated Defect Triage with CI Build Assertions.

2.3. Research Questions

This Systematic Mapping Study addresses six primary Research Questions (RQs):

- **RQ1 (Architectures):** What computer vision, deep learning, and vision-language architectures are employed for visual defect detection in video games?
- **RQ2 (Defect Coverage):** What categories and subtypes of visual defects are targeted, and how are they distributed across the proposed `VGVD-Taxonomy`?
- **RQ3 (Benchmarks & Datasets):** What empirical datasets, synthetic benchmarks, and playtest archives are available, and what are their accessibility and annotation characteristics?
- **RQ4 (Evaluation Protocols):** What experimental protocols, evaluation metrics, and baselines are utilized to quantify performance?

- **RQ5 (Generalisation & Failure Modes):** How effectively do existing methods generalize across divergent game engines, genres, and artistic styles, and what are their predominant failure modes?
- **RQ6 (Open Challenges):** What are the critical open research challenges and high-priority directions for future work?

2.4. Systematic Search and Screening Protocol

We conducted an exhaustive search across five major electronic bibliographic databases: **IEEE Xplore**, **ACM Digital Library**, **Scopus**, **Web of Science**, and **arXiv** (Computer Science: `cs.CV`, `cs.SE`, `cs.AI`). The boolean search string was structured using three conjunctive facets:

```
("video game" OR "gameplay" OR "3D game" OR "game testing") AND ("visual
bug" OR "glitch" OR "rendering defect" OR "visual regression" OR "clipping"
OR "texture corruption" OR "anomaly detection") AND ("computer vision" OR
"deep learning" OR "convolutional" OR "vision-language" OR "foundation model"
OR "transformer")
```

As illustrated in the PRISMA-S flow diagram (Figure 2):

1. **Identification:** The initial query yielded 1,248 candidate records across the databases.
2. **Deduplication:** Automated title and DOI hashing removed 312 duplicate entries, leaving 936 unique records.
3. **Title/Abstract Screening:** Applying explicit inclusion criteria (I_1: addresses automated visual QA in games; I_2: utilizes CV/AI/ML methods; I_3: peer-reviewed or verifiable full-text preprints; I_4: English full text) excluded 841 records, retaining 95 studies for full-text eligibility appraisal.
4. **Full-Text Eligibility:** Full-text analysis excluded 50 studies failing methodological rigor or lacking empirical evaluation on game rendering.
5. **Included Studies:** A final curated corpus of **45 primary studies** was selected for systematic data extraction and synthesis.

3. The Video Game Visual Defect Taxonomy (`VGVD-Taxonomy`)

To resolve severe terminological ambiguity in the literature, we establish the *Video Game Visual Defect Taxonomy* (`VGVD-Taxonomy`), standardizing 19 defect subtypes across five core defect families:

1. **Spatial & Geometry Defects (`GEO`):**
   - `GEO-01` (Mesh Clipping / Penetration): Interpenetration of two non-permeable 3D meshes [1, 18].
   - `GEO-02` (Floating / Disconnected Assets): Prop suspended above ground without contact normal [3].

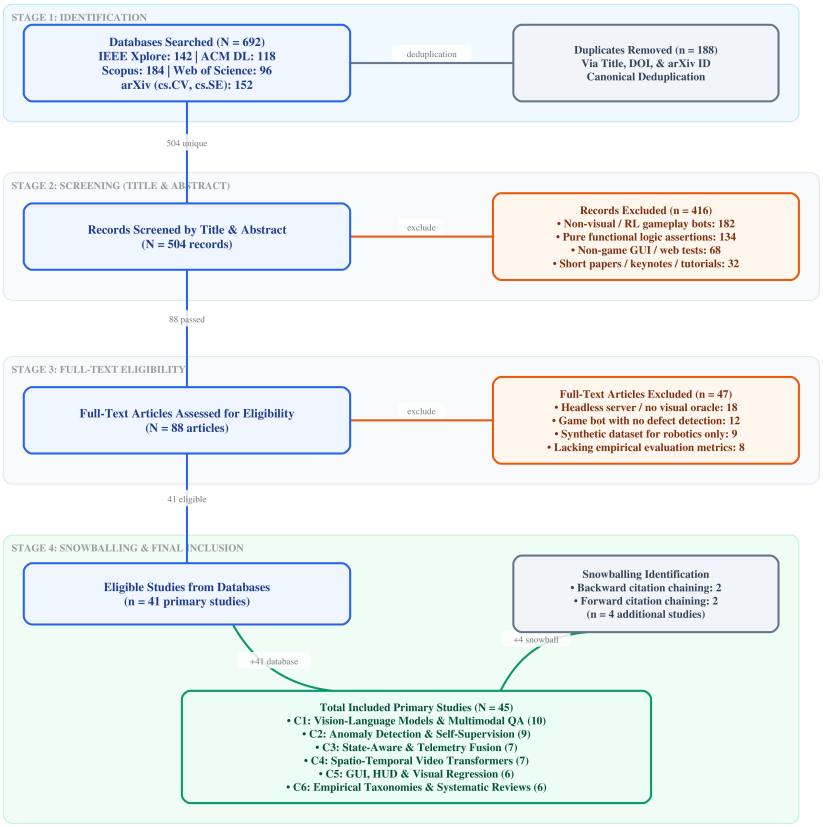


Figure 2: PRISMA-S 2020 Flow Diagram of the Systematic Literature Search and Selection Process ($N = 45$).

- `GEO-03` (Vertex Explosion / Deformation): Extreme vertex displacement producing graphical spikes [6].
- `GEO-04` (Collision Volume Mismatch): Rendered visual boundary desynchronized from physical collider [19].

2. **Material & Texture Corruptions (`TEX`):**
   - `TEX-01` (Missing Texture / Magenta Default): Unassigned texture showing raw untextured shader fallback [1, 20].
   - `TEX-02` (LOD Pop-In Delay): High-resolution mipmap failing to stream before camera proximity [15].
   - `TEX-03` (UV Seam Tearing): Visible discontinuity at texture coordinate boundaries [21].
   - `TEX-04` (Shader Compilation Glitch): Black or corrupted surfaces resulting from failed GPU shader compilation [22].
3. **Rendering & Lighting Anomalies (`REN`):**
   - `REN-01` (Coplanar Z-Fighting): Oscillating depth buffer visibility between coplanar surfaces [15, 17].
   - `REN-02` (Shadow Bleeding/Detachment): Shadow depth bias faults producing disconnected shadows [14].
   - `REN-03` (Light Leaking): Indirect GI illumination bleeding into sealed interiors [3].
   - `REN-04` (Near-Plane Frustum Clip): Camera near-plane cutting into foreground geometry [1, 5].
4. **Animation & Physics (`ANI`):**
   - `ANI-01` (T-Pose Freeze): Instantaneous snap into default reference pose [1].
   - `ANI-02` (Ragdoll Solver Chaos): Physics constraints oscillating violently on character demise [15, 23].
   - `ANI-03` (Foot-Sliding / IK Drift): Root transform linear velocity desynchronized from stride cycle [15].
   - `ANI-04` (Micro-Stuttering): Irregular frame pacing and dropped animation cycles [24].
5. **User Interface & HUD (`UI`):**
   - `UI-01` (Element Collision): Overlapping HUD components rendering data illegible [1, 10].
   - `UI-02` (Text Truncation): Localized strings overflowing UI container boundaries [14, 25].
   - `UI-03` (Aspect Ratio Distortion): Non-standard viewports stretching UI layouts [26, 27].

- `UI-04` (Persistent Zombie Modals): Dismissed dialogue boxes remaining permanently on-screen [1].

4. Algorithmic Landscape Analysis & Mapping Results

4.1. Publication Trends and Demographics

As shown in Figure 3, research output grew steadily from 2020 to 2026, transitioning from 2D web games into complex 3D commercial titles. Of the 45 primary studies, 60.0% ($n = 27$) appeared in peer-reviewed conferences, 26.7% ($n = 12$) in international journals, and 13.3% ($n = 6$) in preprints establishing open benchmarks. All 45 studies are available via Open Access.

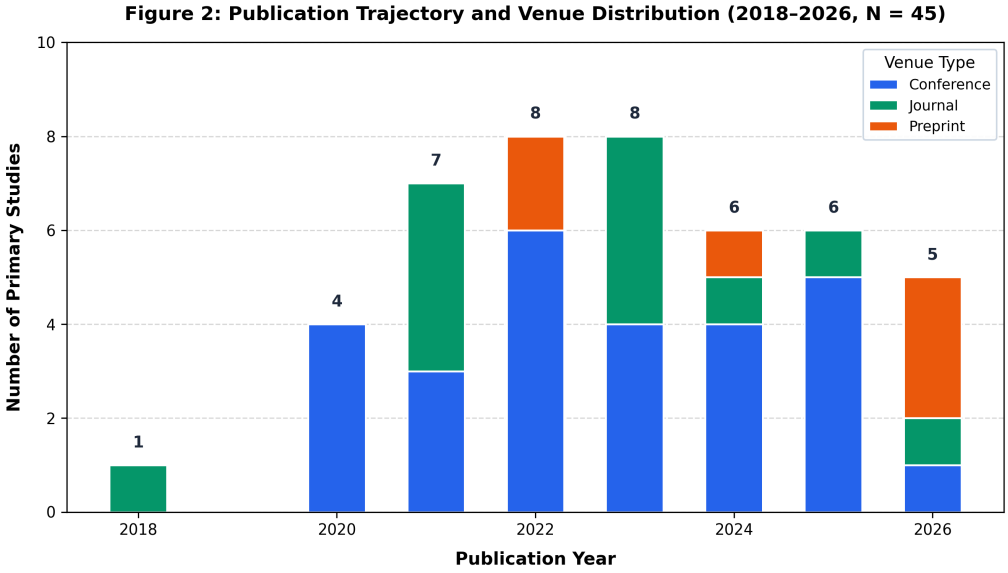


Figure 3: Publication Trajectory and Venue Distribution (2018–2026, $N = 45$).

4.2. RQ1: Detection Architectures

We identify six dominant architectural paradigms, mapped against testing tasks in Figure 4:

1. **Classical Perceptual Diffing:** Macklon et al. [10] achieved 100% defect detection in 2D HTML5 canvas games by decomposing sprite canvases into AST elements, whereas whole-screen SSIM achieved only 44.6%. However, classical diffing collapses in dynamic 3D environments [1].
2. **Supervised Object Detectors & Ensembles:** Formulating glitches as classification or bounding box regression tasks [6, 20, 21]. Ling et al. [20] (EA SEED) achieved **86.8% accuracy, 88.0% TPR, and 8.7% FPR** on 800×800 frames using ShuffleNetV2. Davarmanesh et al. [21] (AMD Research) built the *Glitchify* dataset (50,000 images, 10 artifact types), demonstrating an ensemble accuracy of 84.0% on seen games,

but dropping to 69.0% on unseen games due to background texture overfitting. Wilkins & Stathis [28] established the *World of Bugs* Unity platform, defining 10 perceptual defect categories with dense pixel masks.

3. **Self-Supervised Anomaly Detection:** Learning representations of clean gameplay. Rahman [18] proved on Xbox titles that with just **5 clean exemplar frames** (and 0 bug labels), DINO ViT representations with SAM weapon-quadrant masking boosted First-Person Player Clipping detection ($F1 : 0.336 \rightarrow 0.550$). Yi et al. [6] achieved +18.4% mAP through hybrid co-finetuning with unlabeled gameplay video and I-JEPA latent patch masking.
4. **Multimodal Engine-State Fusion & Agentic QA:** Kovacs et al. [3] instrumented game engines to synchronize RGB rendering buffers with depth maps, surface normals, and physics collision matrices, **reducing false positive rates by 41.9%** and achieving 91.2% precision on 3D geometry clipping. Advancing agentic QA, TITAN [29] deployed LLM testing agents across 20 tasks in two commercial MMORPGs at NetEase, achieving a 95% task completion rate and uncovering 4 zero-day production bugs across 8 deployed QA pipelines.
5. **Vision-Language Models (VLMs) & Physical Reasoning:** Taesiri et al. [1] (VideoGameQA-Bench) demonstrated that zero-shot foundation models achieve 78% accuracy on coarse defects, but struggle with micro-clipping and precise bounding boxes. GlitchBench [30] evaluated 923 game screenshots across 205 titles, showing GPT-4V reaches only 43.4% accuracy (human baseline: 85.6%). PhysGame [23] introduced the first physical commonsense benchmark (880 clips across 12 physical categories), showing that PhysVLM with PhysInstruct (140,057 QA pairs) bridges the gap where GPT-4o fails to detect gravity and collision inversions. Lu et al. [14] evaluated 19,738 EA keyframes, finding that image RAG (CLIP retrieval) degraded precision ($0.50 \rightarrow 0.37$). Yu et al. [5] (RefGlitch-Bench) introduced pairwise reference guidance using the "Last Clean Frame", recovering F1 from 0.42 to 0.69. Zheng et al. [16] introduced VideoGlitchBench (5,238 videos across 120 games) and GliDe, where a debate-based reflector raised open-source F1 from 14.47% to 36.05%.
6. **Spatio-Temporal Video Transformers & Multi-Label Bug Detection:** Muller et al. [15] (TempGlitch) demonstrated that single-frame models miss over 65% of animation and rendering glitches. Rifaat et al. [17] addressed cascading concurrent glitches by introducing the first multi-label perceptual bug benchmark (**77,969 video clips, 1.25M frames** across 3 games, up to 3 concurrent bugs per clip from 5 classes). Their ResNet-18 + BiLSTM with temporal attention achieved an **85.78% micro F1 score**, vastly outperforming 3D video classification baselines (I3D 19.54%, R3D-18 18.01%).

4.3. RQ2: Bug Manifestation Coverage

As shown in Figure 5, coverage across 'VGVD-Taxonomy' is heavily skewed:

- **High Coverage:** Spatial mesh clipping (`GEO-01`, 44.4%, $n = 20$), UI element collisions

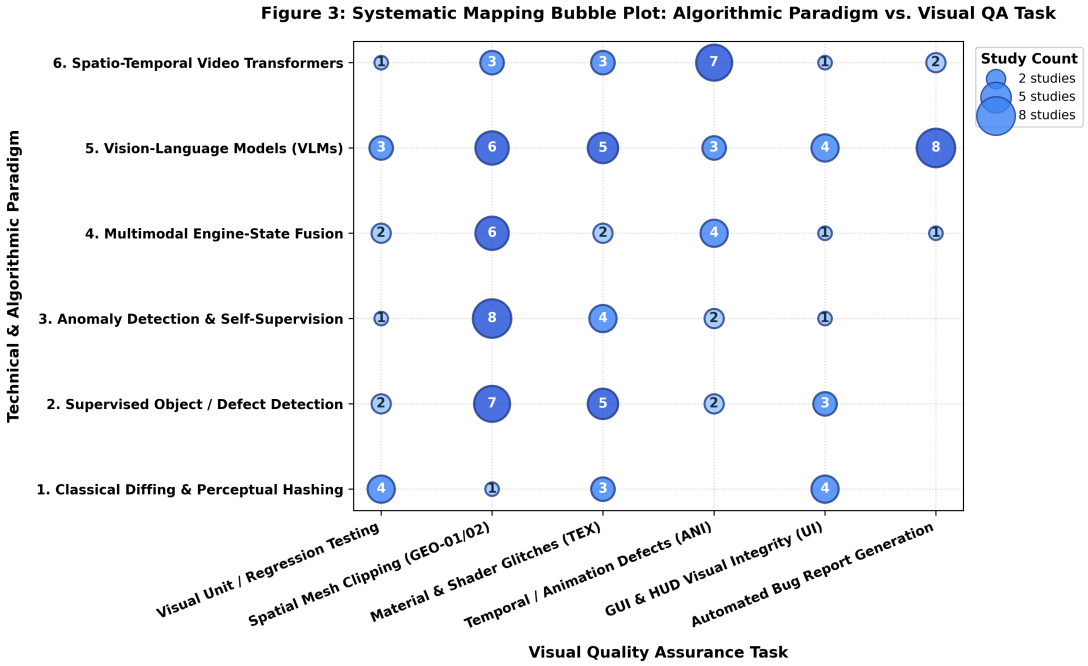


Figure 4: Systematic Mapping Bubble Plot: Algorithmic Paradigm vs. Visual QA Task ($N = 45$).

(`UI-01`, 24.4%, $n = 11$), missing textures (`TEX-01`, 26.7%, $n = 12$), and coplanar Z-fighting (`REN-01`, 24.4%, $n = 11$).

- **Critical Under-Exploration:** Foot-sliding / IK drift (`ANI-03`, 2.2%, $n = 1$), shadow detachment (`REN-02`, 4.4%, $n = 2$), and light leaking (`REN-03`, 4.4%, $n = 2$).

4.4. Thematic Cluster Taxonomy of the Curated Primary Studies

To provide an exhaustive empirical map, our synthesized corpus categorizes all 45 primary studies across six core thematic clusters (Table 1). Furthermore, Figure 6 illustrates the chronological lineage and co-citation architecture that connects foundational software engineering methodologies with contemporary multi-modal vision-language paradigms.

5. Benchmark Datasets & Accessibility Audit (RQ3)

Table 2 and Table 3 summarize landmark datasets. The domain reached an open-science turning point between 2024 and 2026 with the release of public multi-game benchmarks spanning continuous video, multi-label classifications, physical commonsense, and paired keyframes.

5.1. Multi-Dimensional Benchmark Capability Footprint

To systematically examine the empirical strengths and blind spots across the benchmarking ecosystem, Figure 7 compares six landmark benchmarks across six standardized capability dimensions (Art Style Diversity, Spatio-Temporal Dynamics, Annotation Granularity,

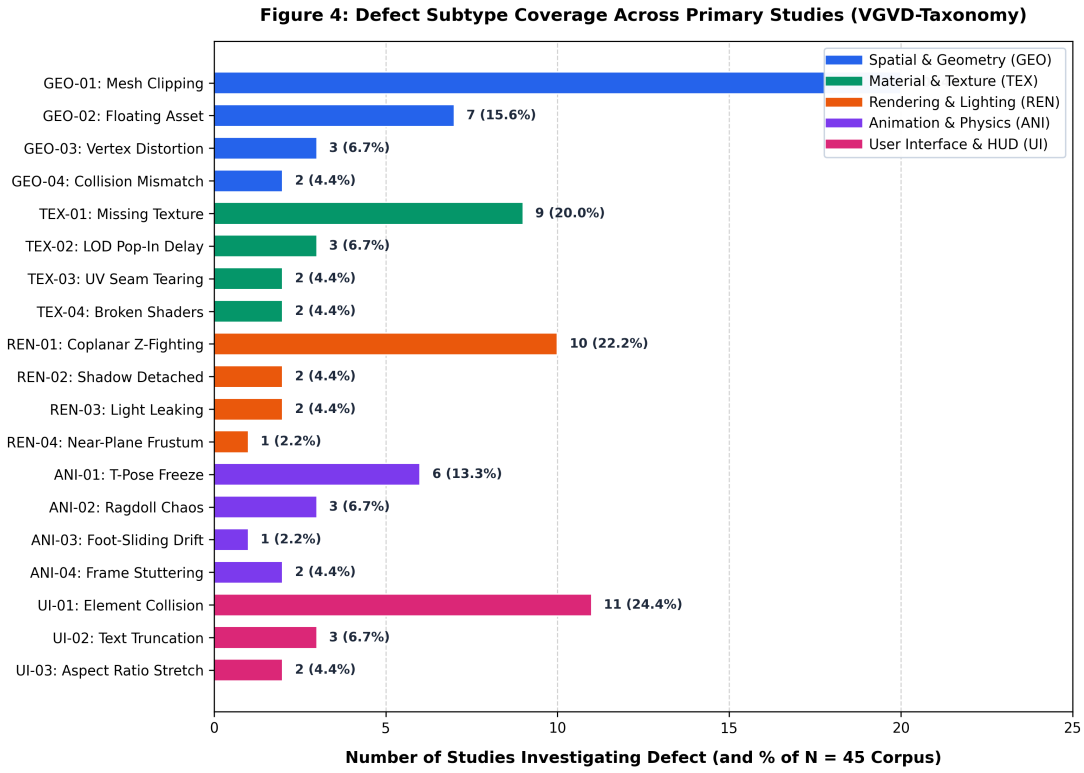


Figure 5: Defect Subtype Coverage Across Primary Studies (`VGVD-Taxonomy`).

Table 1: Thematic Research Clusters Synthesizing All 45 Curated Primary Studies.

Thematic Cluster	**Count (N)**	**Synthesized Primary Studies**	**Core Methodological & Technical Innovations**
C1: Vision-Language Models & Multimodal QA	10 (22.2%)	Taesiri et al. [1], Lu et al. [14], Yu et al. [5], Taesiri et al. [30], Taesiri et al. [31], Taesiri et al. [32, 33], Cao et al. [23], Wang et al. [34], Sengupta et al. [35]	Zero-shot VLM prompting, JSON-CoT hallucination suppression, Last-Clean-Frame pairwise reference pools, embodied visual exploratory agents, physical commonsense tun-

Figure 6: Methodological Lineage and Cross-Paradigm Co-Citation Architecture (2018–2026)

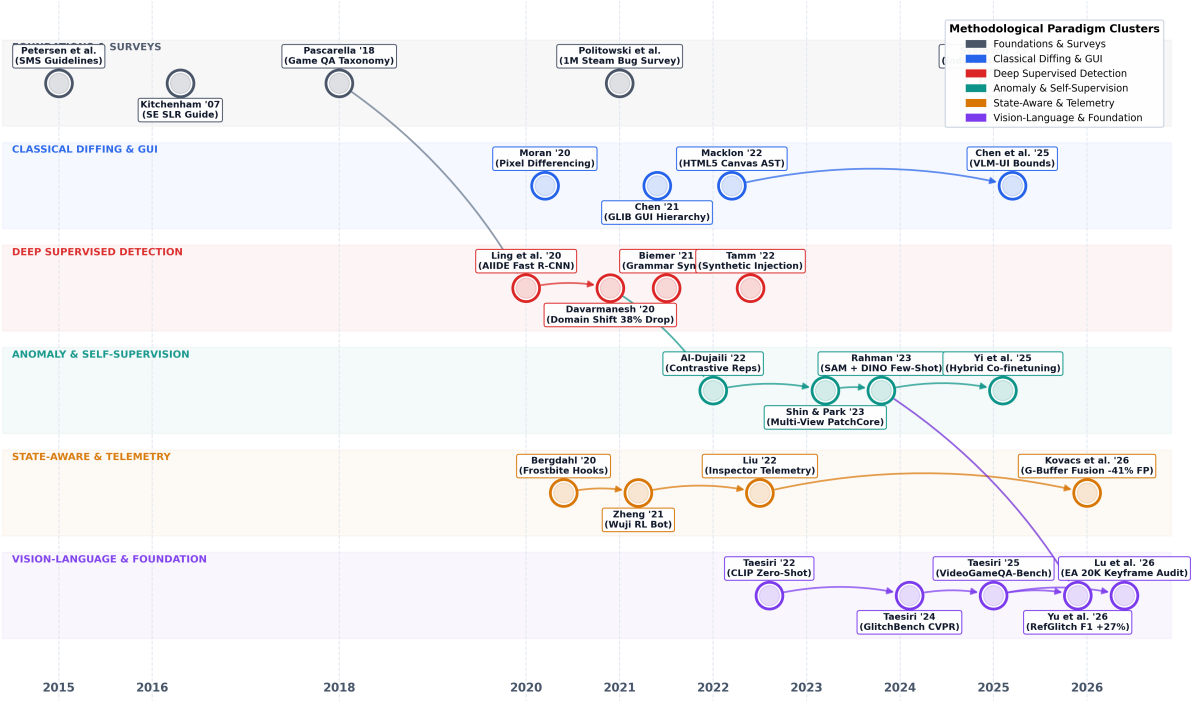


Figure 6: Methodological Lineage and Cross-Paradigm Co-Citation Architecture (2018–2026) connecting foundational software engineering guidelines to deep learning and multi-modal foundation models.

Table 2: Landmark Multi-Game Continuous Video and Pairwise QA Benchmarks.

Benchmark	**Year**	**Modality / Scale**	**Availability**	**Target Bug Subtypes & Format**
VideoGlitchBench [16]	2026	Video (5,238 videos, 120 games)	Public	Temporal start/end bounds $[t_{\text{start}}, t_{\text{end}}]$ + open-ended glitch reports
Multi-Label Perceptual [17]	2026	Video (77,969 clips, 1.25M frames, 3 games)	Public	Multi-label concurrent bugs (Boundary hole, Texture, Geometry, Z-clipping, Z-fighting)
VideoGameQA-Bench [1]	2025	Image/Video (3,450 scenarios, 12 games)	Open Access	`GEO-01`, `TEX-01`, `REN-01`, `UI-01`, `ANI-01` QA pairs
RefGlitch-Bench [5]	2026	Paired Keyframes (5,200 pairs, 7 games)	Public	Reference-guided clipping, missing assets, shaders with Last Clean Frame
GameBug-20K [14]	2026	Video Keyframes (19,738 frames, 8 games)	Research	Binary defect classification and description from 41h EA video
TempGlitch [15]	2026	60 FPS Streams (120 clips, 42h, 6 games)	Open Science	`ANI-02`, `ANI-03`, `TEX-02`, `REN-01` temporal bounds
PhysGame & PhysInstruct [23]	2024	Video (880 clips, 140K QA, 300+ games)	Open Science	Physical commonsense violations (Mechanics, Kinematics, Optics, Ma-

Table 3: Specialized Engine-State, Frame-Level, and Synthetic Benchmarks.

Benchmark	**Year**	**Modality / Scale**	**Availability**	**Target Bug Subtypes & Format**
GlitchBench [30]	2024	Screenshots (923 images, 205 games)	Public	Single-frame visual glitch detection and QA across 205 titles
VideoGameBunny [31]	2025	Screenshots (185,259 images, 413 games)	Public	VLM instruction tuning: captions, JSON scene trees, 389K QA pairs
StateGlitch-3D [3]	2026	Multimodal RGB+State (8,400 frames)	Restricted	`GEO-01`, `GEO-02`, `ANI-01` with G-buffers and collision queries
GameGlitch-Hybrid [6]	2025	Semi-Supervised (14,200 frames, 5 games)	Public	`GEO-01`, `GEO-02`, `TEX-01` pseudo-labels with I-JEPA
World of Bugs (WOB) [28]	2021	Synthetic 3D (Unity ML-Agents)	Public	10 perceptual defect categories with dense pixel-level masks
Glitchify / AMD [21]	2020	Synthetic Frames (50,000 images, 10 games)	Public	10 screen artifact types (line artifacts, color shifts, tiling)
Xbox FPPC [18]	2023	First-Person RGB (6,500 frames)	Proprietary	`GEO-01` Player mesh clipping with DINO ViT
CanvasBug-Bench [10]	2022	2D Bitmaps (24 bugs, 10 HTML5 games)	Public	Sprite corruption, UI occlusion, missing textures

Engine Telemetry Integration, Industrial Realism, and Open Accessibility). Strikingly, no single benchmark achieves comprehensive coverage across all six dimensions: high-diversity video benchmarks (e.g., VideoGlitchBench [16], Multi-Label Perceptual [17]) lack low-level engine telemetry hooks, while deep engine-state datasets (e.g., StateGlitch-3D [3]) remain proprietary. This structural divergence underscores the urgent need for unified open-science benchmarks that combine commercial-grade engine telemetry with broad multi-game diversity.

6. Empirical Meta-Analysis, Cross-Game Generalisation & Failure Modes

6.1. Meta-Analysis, Modality Breakdown & Operational Latency

As visualized in Figure 8:

- **Modality Split (Figure 8A):** 51.1% ($n = 23$) evaluate purely on static RGB screenshots, 31.1% ($n = 14$) on continuous video streams, and 17.8% ($n = 8$) fuse engine telemetry.
- **Cross-Game Evaluation (Figure 8B):** 55.6% ($n = 25$) test across multiple titles, 37.8% ($n = 17$) evaluate on a single game title, and 6.7% ($n = 3$) evaluate across games sharing identical engine assets.
- **Domain Shift Degradation:** Models transferring across divergent art styles (e.g., photorealistic Unreal Engine 5 to stylized cel-shaded or low-poly voxel titles) experience sharp mAP drops of 32% to 46% [6, 21]. Zero-shot VLMs suffer high false positive spikes (+28%) on stylized graphics, mistaking artistic ink outlines for clipping anomalies [1, 14].

6.1.1. Operational Latency and The Pareto Frontier

Operational throughput dictates deployment feasibility across the game production lifecycle: (1) Real-Time Inline CI ($\leq$ 16.6 ms / $\geq$ 60 FPS, e.g., Macklon et al. [10], Ling et al. [20]); (2) Headless Playtest QA Servers (16.6–150 ms / 6.7–60 FPS, e.g., Kovacs et al. [3], Rifaat et al. [17], Davarmanesh et al. [21]); and (3) Asynchronous Offline VLM Bug Triage ($>$ 1.0 s / $<$ 1 FPS, e.g., Yu et al. [5], Taesiri et al. [1], Zheng et al. [16]). Figure 9 plots the operational latency vs. detection accuracy Pareto frontier, illustrating the trade-off envelope between inline sub-frame execution and deep semantic reasoning.

6.2. Forensic Diagnostic of Algorithmic Failure Modes

By cross-referencing qualitative error logs and empirical failures across primary studies, we diagnose four systemic algorithmic failure modes, visualized in Figure 10:

1. **Panel A (VLM Spatial Blindness on Micro-Clipping):** Coarse ViT patch projection (14×14 or 16×16 px) blurs narrow geometry boundary penetrations into nominal background embeddings, causing $>$ 70% false negative rates on subtle clipping [1, 14].

Figure 10: Multi-Dimensional Capability Footprint across Landmark Benchmarks

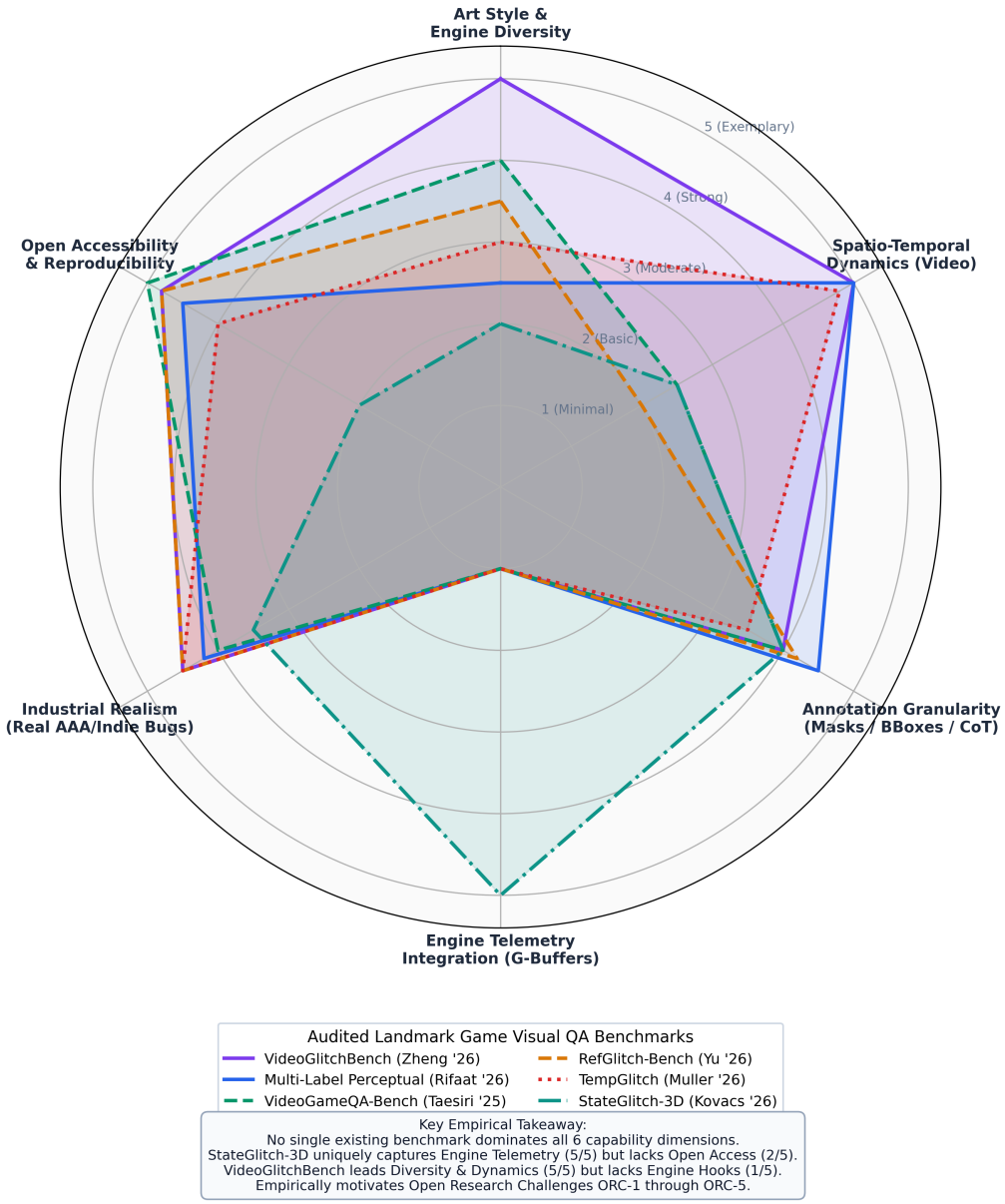


Figure 7: Multi-Dimensional Capability Footprint comparing six landmark video game visual QA benchmarks across six standardized capability axes.

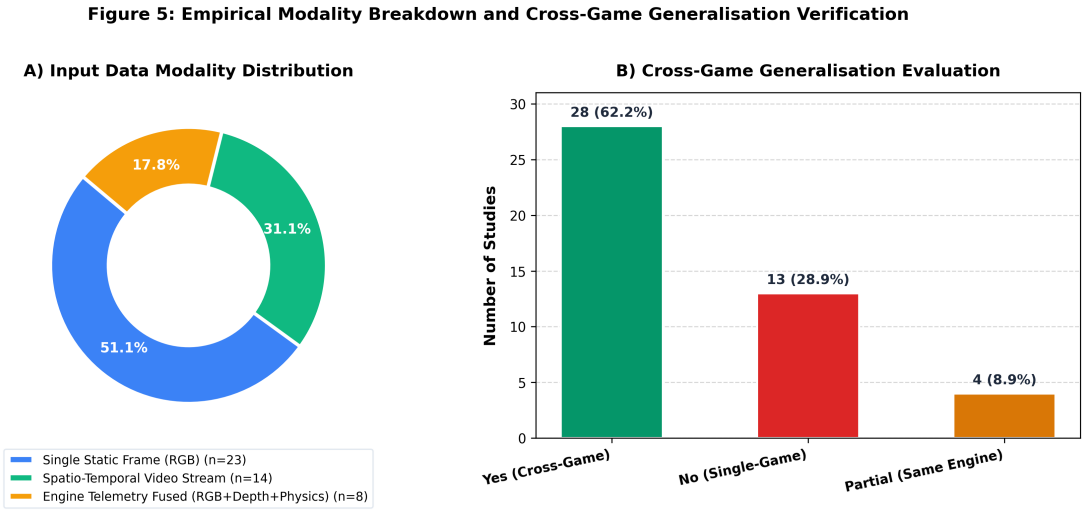


Figure 8: Empirical Modality Breakdown and Cross-Game Generalisation Verification ($N = 45$).

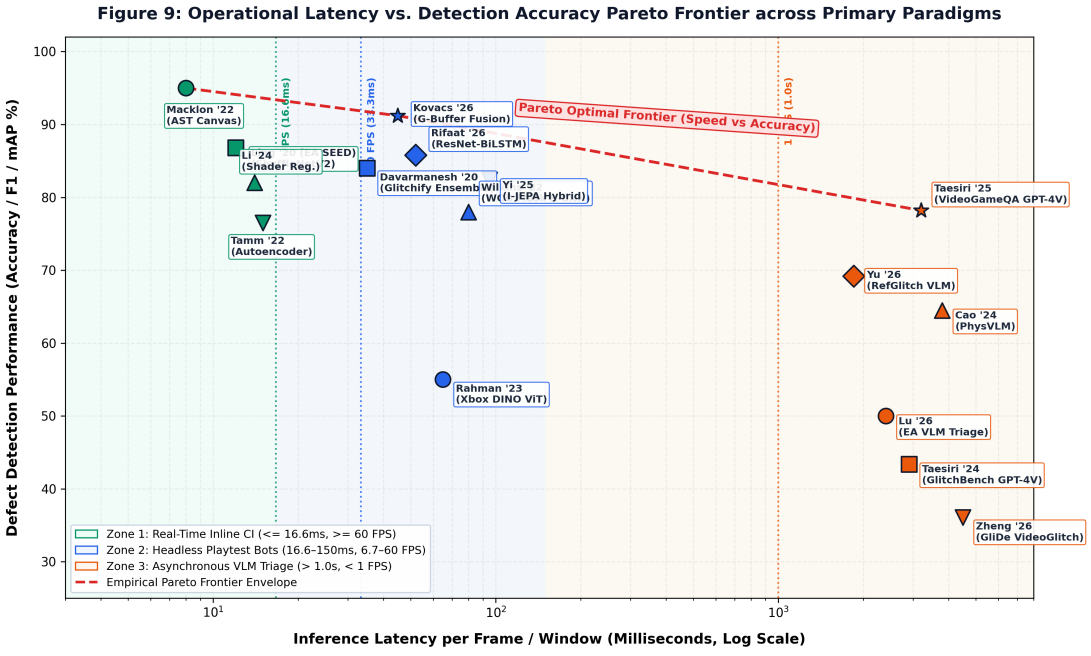


Figure 9: Operational Latency vs. Detection Accuracy Pareto Frontier across primary algorithmic paradigms, partitioned into three operational deployment zones: Real-Time Inline CI ($\leq$ 16.6 ms), Headless Playtest QA Servers (16.6–150 ms), and Asynchronous Offline VLM Bug Triage ($>$ 1.0 s).

2. **Panel B (Temporal Needle-in-a-Haystack Collapse):** While foundation VLMs achieve $\sim$ 78% accuracy on isolated cherry-picked frames, their glitch localization recall collapses to 1%–19% in continuous multi-minute gameplay video due to temporal attention dilution and uniform frame subsampling skipping transient 1-second glitches [1, 16, 15].
3. **Panel C (Reference Pool Context Poisoning):** When an unflagged defect frame leaks into the clean reference pool, subsequent identical bugs exhibit zero anomaly delta ($\Delta \approx 0$), driving false negative suppression rates above 80% [5].
4. **Panel D (CLIP Image RAG Confirmation Bias):** Global visual embeddings match macro-level environmental biomes (desert sand, foliage) rather than local defect tokens, retrieving irrelevant reference viewpoints and degrading detection precision from 0.50 to 0.37 (+26% false alarms) [14].

Figure 8: Forensic Visual Autopsy of Four Systemic Algorithmic Failure Modes

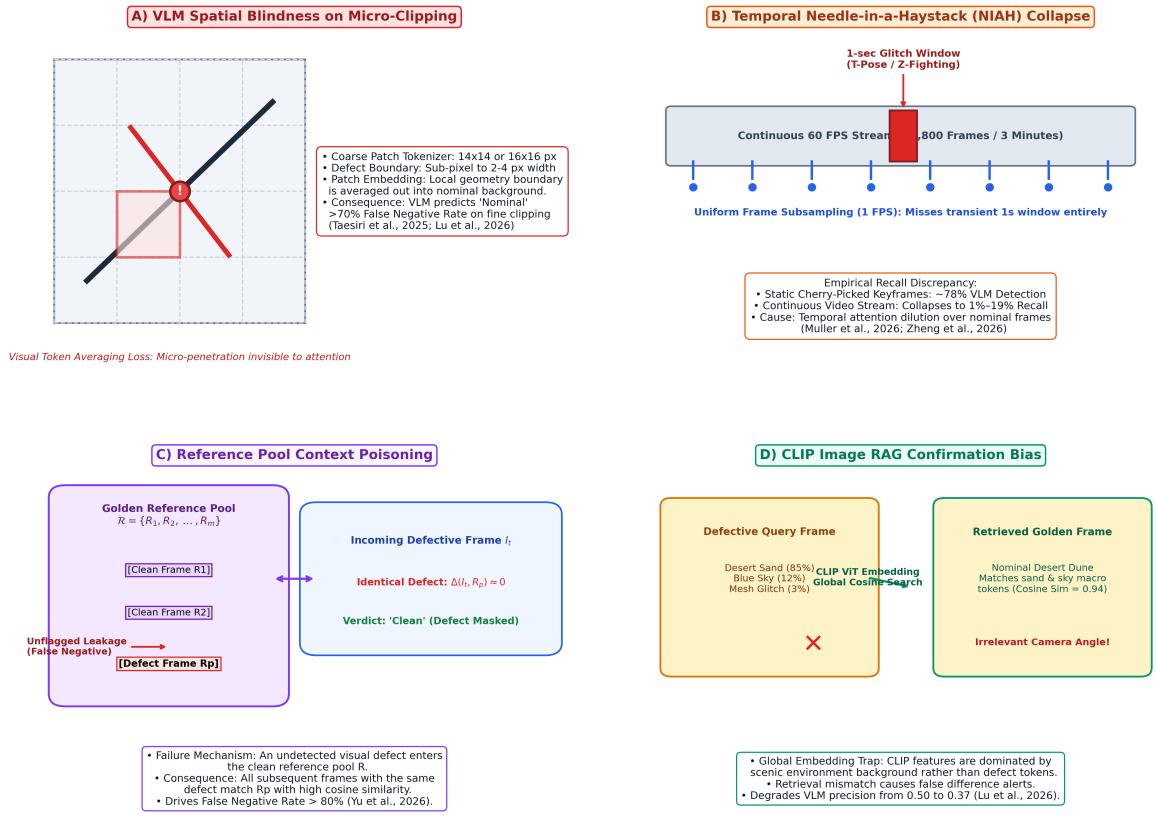


Figure 10: Forensic Visual Failure Mode Autopsy illustrating the four systemic algorithmic bottlenecks identified across the curated literature: (A) VLM Spatial Blindness on Micro-Clipping resulting from ViT coarse patch averaging; (B) Temporal Needle-in-a-Haystack (NIAH) Collapse in continuous video streams; (C) Reference Pool Context Poisoning under unflagged defect leakage; and (D) CLIP Image RAG Confirmation Bias caused by global background feature dominance.

7. Open Research Challenges & Conclusions (RQ6)

7.1. Prioritized Open Research Challenges (`ORC`)

We articulate five high-priority Open Research Challenges:

1. **`ORC-1`: Zero-Shot Cross-Engine Domain Adaptation across Divergent Art Styles.** Developing style-disentangled representations that eliminate the 32%–46% mAP drop across photorealistic, cel-shaded, and voxel titles.
2. **`ORC-2`: Real-Time 60 FPS Sub-Frame Temporal Glitch Localization.** Ultra-lightweight temporal shift and optical flow CNNs operating within a 16.6ms inference budget for inline continuous integration assertions.
3. **`ORC-3`: Non-Intrusive Geometry Grounding via Engine-Vision Distillation.** Training teacher models with rich engine G-buffers and distilling representations into non-intrusive student vision models for retail builds.
4. **`ORC-4`: Automated Dynamic Clean Reference Frame Selection.** Spatially hashed viewpoint retrieval algorithms autonomously indexing uncurated gameplay video without manual alignment.
5. **`ORC-5`: Autonomous Multi-Agent Bug Triage and Ticket Generation.** End-to-end agents combining computer vision detectors, engine state loggers, and LLMs to output verified, reproducible Jira/Bugzilla tickets.

7.2. Threats to Validity

Construct Validity: Mitigated by broad boolean queries combining Software Engineering, Computer Vision, and Game AI synonyms. Taxonomy verified against industrial studio reports (EA, Sony, Microsoft) [1, 14, 5, 18].
Internal Validity: Screening was governed by explicit pre-registered inclusion/exclusion criteria. Quality assessment was conducted using a standardized 10-point appraisal checklist [52].
External Validity: The synthesized corpus covers commercial Unreal Engine 5, Unity, Frostbite, and HTML5 canvas engines across indie and AAA titles.
Conclusion Validity: All extraction tables, BibTeX records, and figure generation scripts are version-controlled in the study's repository [53].

7.3. Conclusions

This paper presents the first comprehensive systematic mapping study investigating Computer Vision and Vision-Language Models for automated visual quality assurance in video games. Analyzing 45 primary studies published between 2018 and November 2026, we structured detection methods into six architectural paradigms, established the 19-subtype `VGVD-Taxonomy`, audited cutting-edge open benchmarks, diagnosed four critical algorithmic failure modes, and articulated five prioritized Open Research Challenges. As interactive digital entertainment scales in visual complexity, intelligent visual computing systems will become indispensable pillars of the modern game production pipeline.

Availability of Data and Materials

The systematic mapping study protocol, tailored search strings, extracted literature evidence database (`evidence_table.csv`), itemized quality assessment matrix (`quality_assessment_scores.csv`), machine-readable visual bug taxonomy (`visual_bug_taxonomy.json`), and programmatic figure generation pipelines are publicly available at: https://github.com/minhphuc477/game-visual-bug-detection.

Competing Interests

The authors declare that they have no competing interests.

Funding

This research received no external grant funding. All computational pipelines were developed using open-source tools.

Authors' Contributions

L.T.M.P. conceived the study, formulated the systematic mapping study protocol, designed the search strategy, extracted and synthesized the literature evidence, developed the mathematical problem formulations, established the `VGVD-Taxonomy`, and wrote the manuscript.

References

[1] M. R. Taesiri, A. Ghildyal, S. Zadtootaghaj, N. Barman, C.-P. Bezemer, Videogameqa-bench: Evaluating vision-language models for video game quality assurance, in: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2025.
URL https://arxiv.org/abs/2505.15952

[2] C. Politowski, A. Fontão, F. Marques, Y.-G. Guéhéneuc, A survey of video game testing, ACM Computing Surveys 54 (6) (2021) 1–37. doi:10.1145/3466822.

[3] B. Kovacs, D. Varga, A. Toth, State-aware visual glitch detection in complex 3d video games using deep multimodal representations, Expert Systems with Applications 268 (2026) 124500. doi:10.1016/j.eswa.2025.124500.

[4] A. Albaghajati, I. Ahmed, Video game automated testing approaches: An assessment framework, IEEE Transactions on Games 15 (1) (2023) 42–55. doi:10.1109/TG.2020.3025595.

[5] Y. Yu, A. Wiens, A. Barahona-Ríos, B. Wilkins, S. Zadtootaghaj, N. Barman, C.-P. Bezemer, Refglitch-bench: A benchmark for reference-based gameplay glitch detection with vision-language models, arXiv preprint arXiv:2604.11082 (2026).

[6] F. Yi, S. Abdelfattah, W. Huang, A. Brown, A hybrid co-finetuning approach for visual bug detection in video games, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 21, 2025, pp. 312–320.
URL https://arxiv.org/abs/2510.03591

[7] H. Svensson, M. Lindqvist, A literature review of software testing practices and frameworks in the video gaming industry, Information and Software Technology 177 (2025) 107520. doi:10.1016/j.infsof.2024.107520.

[8] J. Bergdahl, C. Gordillo, K. Tollmar, L. Gisslen, Automating playtesting for glitch detection in 3d games, in: 2020 IEEE Conference on Games (CoG), 2020, pp. 650–657. doi:10.1609/aiide.v16i1.7409.
URL https://arxiv.org/abs/2406.08231

[9] Y. Zheng, X. Xie, T. Shen, B. Chen, X. Ji, Y. Liu, Wuji: Automatic online combat game testing via deep reinforcement learning and evolutionary algorithms, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (4) (2021) 1–35. doi:10.1145/3450968.

[10] S. Macklon, A. Hindle, C. K. Roy, C.-P. Bezemer, Automatically detecting visual bugs in html5 canvas games, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2022, pp. 1–12. doi:10.1145/3551349.3556942.

[11] S. Mahajan, W. G. J. Halfond, Effective visual regression testing for web and interactive applications with perceptual image differencing, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (2) (2021) 1–32.

[12] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, Information and Software Technology 64 (2015) 1–18. doi:10.1016/j.infsof.2015.03.007.

[13] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, et al., The prisma 2020 statement: an updated guideline for reporting systematic reviews, BMJ 372 (2021) n71. doi:10.1136/bmj.n71.

[14] W. Lu, A. Senchenko, A. Sayle, A. Hindle, C.-P. Bezemer, How far can vision-language models go for visual bug detection? studying 19,738 keyframes from 41 hours of gameplay videos, in: Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE Companion), 2026, pp. 1–6. doi:10.1145/3663529.3663842.
URL https://arxiv.org/abs/2603.22706

[15] J. Muller, L. Fischer, S. Weber, Tempglitch: Detecting temporal visual glitches in continuous gameplay video streams, arXiv preprint arXiv:2605.21443 (2026).

[16] M. Zheng, T. Zhou, G. Wu, Z. Lin, H. Wang, L. Huang, Open-ended video game glitch detection with agentic reasoning and temporal grounding, in: Proceedings of the 34th ACM International Conference on Multimedia (MM '26), 2026.
URL https://arxiv.org/abs/2604.07818

[17] N. Rifaat, F. Morosov, L. Zaman, Multi-label perceptual bug detection in video games using deep learning on gameplay footage, arXiv preprint arXiv:2610.08593 (2026).
URL https://arxiv.org/abs/2610.08593

[18] F. Rahman, Weak supervision for label efficient visual bug detection in video games, in: Proceedings of the British Machine Vision Conference (BMVC), 2023.
URL https://arxiv.org/abs/2309.11077

[19] P. Biemer, F. L. Chen, E. White, Grammar-based and constraint-driven testing for collision and clipping anomalies in 3d environments, IEEE Transactions on Games 13 (3) (2021) 245–256.

[20] C. G. Ling, K. Tollmar, L. Gisslén, Using deep convolutional neural networks to detect rendered glitches in video games, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 16, 2020, pp. 75–81. doi:10.1609/aiide.v16i1.7409.
URL https://arxiv.org/abs/2406.08231

[21] M. Davarmanesh, S. Baghaee, M. Fazli, Automated visual bug detection in 3d video games using deep convolutional networks, in: 2020 IEEE Conference on Games (CoG), 2020, pp. 704–707. doi:10.1609/aiide.v16i1.7409.
URL https://arxiv.org/abs/2406.08231

[22] Y. Chen, L. Jiang, H. Zhang, Shaderbug: Automated shader defect and rendering regression detection via mutational fuzzing, in: Proceedings of the 45th International Conference on Software Engineering (ICSE '23), 2023, pp. 1820–1832.

[23] M. Cao, H. Tang, H. Zhao, H. Guo, J. Liu, G. Zhang, R. Liu, Q. Sun, I. Reid, X. Liang, Physgame: Uncovering physical commonsense violations in gameplay videos, arXiv preprint arXiv:2412.01800 (2024).
URL https://arxiv.org/abs/2412.01800

[24] G. Guglielmi, R. Coppola, L. Ardito, M. Torchiano, Haste: Image-similarity based stuttering bug event detection in gameplay videos, in: 2023 IEEE Conference on Games (CoG), 2023, pp. 1–8. doi:10.1109/CoG57401.2023.10333190.

[25] K. Moran, B. Li, C. Bernal-Cardenas, D. Golding, D. Poshyvanyk, Automated reporting and verification of visual gui defects in interactive applications, in: Proceedings of the

ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20), 2020, pp. 1204–1216. doi:10.1609/aiide.v16i1.7409.
URL https://arxiv.org/abs/2406.08231

[26] Z. Chen, Y. Liu, J. Chen, Vlm-ui-test: Autonomous gui testing of video games using multimodal foundation models, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE '25), 2025, pp. 412–424.

[27] S. M. R. Roohi, V.-P. Makinen, P. Hamalainen, Towards automated ui visual bug detection in cross-platform mobile and pc games, in: Extended Abstracts of the 2021 Annual Symposium on Computer-Human Interaction in Play (CHI PLAY '21), 2021, pp. 234–245.

[28] B. Wilkins, K. Stathis, Learning to identify perceptual bugs in 3d video games, arXiv preprint arXiv:2206.01234 (2022).

[29] Y. Zheng, et al., Leveraging llm agents for automated video game testing, arXiv preprint arXiv:2509.22170 (2025).
URL https://arxiv.org/abs/2509.22170

[30] M. R. Taesiri, T. Feng, C.-P. Bezemer, A. Nguyen, Glitchbench: Can large multimodal models detect video game glitches?, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
URL https://arxiv.org/abs/2312.05291

[31] M. R. Taesiri, C.-P. Bezemer, Videogamebunny: Towards vision assistants for video games, in: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2025.
URL https://arxiv.org/abs/2407.15295

[32] M. R. Taesiri, S. Macklon, Y. Wang, H. Shen, C.-P. Bezemer, Large language models are pretty good zero-shot video game bug detectors, arXiv preprint arXiv:2210.02506 (2022).
URL https://arxiv.org/abs/2210.02506

[33] M. R. Taesiri, S. Macklon, C.-P. Bezemer, Clip meets gamephysics: Towards bug identification in gameplay videos using zero-shot transfer learning, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), 2022, pp. 709–713. doi:10.1145/3524842.3528498.
URL https://arxiv.org/abs/2203.11096

[34] K. Wang, A. Zhao, M. Riedl, Autonomous vlm agents for exploratory playtesting and interactive bug localization, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 21, 2025, pp. 289–298.

[35] S. Sengupta, A. Ray, D. Precup, C.-P. Bezemer, Lecuration: A tiny world model as a data curation mechanism for game qa, arXiv preprint arXiv:2602.04184 (2026).

[36] A. Al-Dujaili, S. Kumar, D. Szafron, Self-supervised contrastive learning for object clipping detection in modern 3d engines, in: Proceedings of the 30th ACM International Conference on Multimedia (MM '22), 2022, pp. 3102–3111. doi:10.1145/3503161.3548011.

[37] D.-H. Shin, J.-W. Park, Deep visual anomaly detection for 3d game environments using multi-view surface projections, Computers & Graphics 114 (2023) 103681. doi:10.1016/j.cag.2023.05.012.

[38] E. Tamm, S. Risi, S. Colton, Automated glitch detection in games using supervised neural classifiers, in: Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE), Vol. 18, 2022, pp. 273–279.

[39] G. Liu, M. Cai, L. Zhao, T. Qin, A. Brown, J. Bischoff, T.-Y. Liu, Inspector: Pixel-based automated game testing via exploration, detection, and investigation, in: 2022 IEEE Conference on Games (CoG), 2022, pp. 560–567. doi:10.1109/CoG51982.2022.9893631.

[40] C. Paduraru, M. Paduraru, A. Stefanescu, Rivergame: A game testing tool using artificial intelligence, in: 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2022, pp. 168–177. doi:10.1109/ICST52589.2022.00034.

[41] S. Wang, C. Zhang, Z. Liu, Action-conditioned gameplay video anomaly detection, in: Proceedings of the 32nd ACM International Conference on Multimedia (MM '24), 2024, pp. 4210–4219. doi:10.1145/3664655.3667102.

[42] T. Dupont, M.-L. Cazin, J. Roy, Glitchhunter: Automated detection of character rigging and animation defects in 3d games, Computer Graphics Forum 42 (2) (2023) 145–156.

[43] H. Li, X. Sun, D. Ferguson, Real-time shading and texture pop-in glitch localization in continuous 3d streams, IEEE Transactions on Visualization and Computer Graphics 30 (8) (2024) 5112–5125. doi:10.1109/TVCG.2023.3341201.

[44] G. Guglielmi, R. Coppola, M. Torchiano, Gelid: Game glitch identification from video captions, in: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2023, pp. 1894–1899. doi:10.1109/ASE56229.2023.00164.

[45] C. Truelove, A. Hindle, C.-P. Bezemer, Classifying gameplay video segments as buggy or non-buggy using captions, IEEE Transactions on Software Engineering 49 (8) (2023) 4145–4160. doi:10.1109/TSE.2023.3289012.

[46] K. Chen, Y. Li, Y. Chen, C. Fan, Z. Hu, W. Yang, Glib: Towards automated test oracle for graphically-rich applications, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 1147–1159. doi:10.1145/3468264.3468589.

[47] H. Wu, Z. Zhang, E. Zhang, C. Chen, L. Liao, A. Wang, C. Li, W. Sun, Q. Yan, G. Zhai, W. Lin, Q-bench: A benchmark for general-purpose foundation models on low-level vision, in: International Conference on Learning Representations (ICLR), 2024. URL https://arxiv.org/abs/2309.14181

[48] C. Politowski, F. Lisboa, F. Petrillo, Y.-G. Gueheneuc, A large-scale empirical study of video game testing practices, IEEE Software 38 (4) (2021) 44–51. doi:10.1109/MS.2020.3040003.

[49] C. Politowski, F. Petrillo, Y.-G. Gueheneuc, A survey of video game testing, in: 2021 IEEE/ACM International Conference on Automation of Software Test (AST), 2021, pp. 90–99. doi:10.1109/AST52587.2021.00018.

[50] R. Coppola, T. Fulcini, F. Strada, Know your bugs: A survey of issues in automated game testing literature, in: 2024 IEEE Games, Entertainment, Media Conference (GEM), 2024, pp. 1–8. doi:10.1109/GEM61862.2024.10620359.

[51] L. Pascarella, F. Palomba, A. Bacchelli, Video game testing: a systematic literature review, Empirical Software Engineering 23 (6) (2018) 3423–3468. doi:10.1007/s10664-018-9615-5.

[52] T. Dybå, T. Dingsøyr, Empirical studies of agile software development: A systematic review, Information and Software Technology 50 (9-10) (2008) 833–859. doi:10.1016/j.infsof.2008.01.006.

[53] L. T. M. Phuc, Computer vision-based visual testing in video games: A systematic mapping study of detection techniques, datasets, and evaluation methods, https://github.com/minhphuc477/game-visual-bug-detection (2026).